



The Stack Data Structure

A Complete Guide to LIFO, Array-Based & Dynamic Stacks in C

Abstract: A stack is a fundamental linear data structure that enforces the Last-In, First-Out (LIFO) principle — the most recently inserted element is always the first one removed. This handout covers stack theory from first principles, provides a production-grade dynamic array implementation in C, examines four classic implementation pitfalls in depth, and closes with practice problems ranging from string reversal to postfix expression evaluation.

Contents

S.No	Topic	Page
1	Introduction & The LIFO Principle	3
2	Real-Life Analogies	3
3	Visual Diagram — Array-Based Stack Layout	3
4	Array-Based Implementation	4
	◦ Linked-List-Based Stack	6
5	Pitfalls While Solving	7
	◦ The Underflow Dilemma	7
	◦ The Top Pointer Index (-1 vs 0)	7
	◦ Pass-by-Value vs Pass-by-Reference	8
	◦ Static vs Dynamic Scaling	8
6	Debugging & Coding Tips	8
7	Practice Problems	9
	◦ Reverse a String Using a Stack	9
	◦ Check for Balanced Parentheses	9
	◦ Evaluate a Postfix Expression	10
8	Debugging Scenarios	10
9	Competitive Programming Tips	13
	◦ Problem 1: Pass-by-Value Trap	10
	◦ Problem 2: Wrong Top Convention	11
	◦ Problem 3: Realloc Overwrite	11
	◦ Problem 4: Uninitialized Pointer	11
	◦ Problem 5: Read-Before-Check	12

1 Introduction & The LIFO Principle

Definition — Stack

A **stack** is a linear data structure that stores elements in a strict **LIFO (Last-In, First-Out)** order. The most recently inserted element is always the first one to be removed. All insertions and deletions happen at a single end, conventionally called the **top**.

A stack exposes (at minimum) two core operations:

- **push** — insert a new element on top of the stack.
- **pop** — remove and return the element currently on top.

Everything else — **peek**, **isEmpty**, **isFull** — exists to support these two safely.

2 Real-Life Analogies

Analogy — Cafeteria Plates

Picture a spring-loaded stack of plates in a cafeteria. You can only take the plate on top, and you can only add a new plate on top. You cannot pull a plate from the middle without disturbing everything above it. That physical constraint *is* LIFO.

Analogy — The “Undo” Button

Every time you make an edit in a text editor, that action is *pushed* onto an internal undo-stack. Pressing **Ctrl+Z** *pops* the most recent action off that stack and reverts it — always the last thing you did, never something from three edits ago.

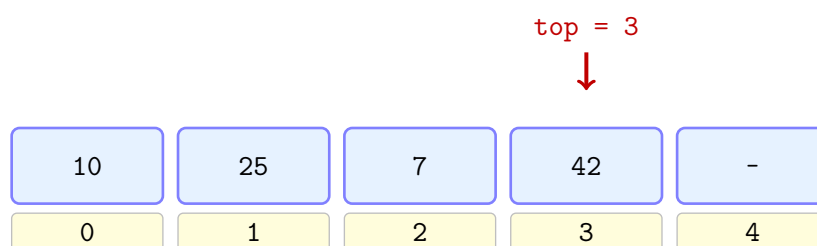
Analogy — Browser History (Back Button)

Each page you visit is pushed onto a history stack. Hitting **Back** pops the most recent page. This is also why “Back” always undoes your *last* navigation, not your first.

3 Visual Diagram — Array-Based Stack Layout

A stack can be implemented on top of a plain array. The **top** variable is simply an index that tracks the position of the most recently pushed element.

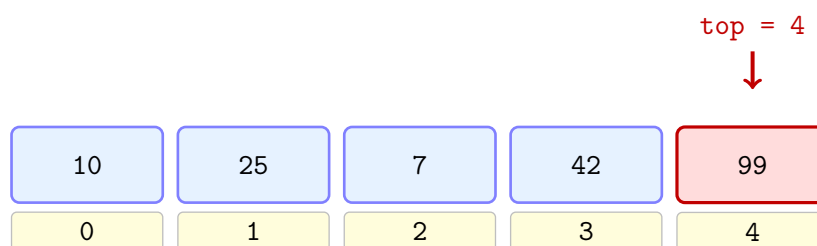
Initial State — Stack of Capacity 5, Holding 4 Elements



capacity = 5

push(99) — Step by Step

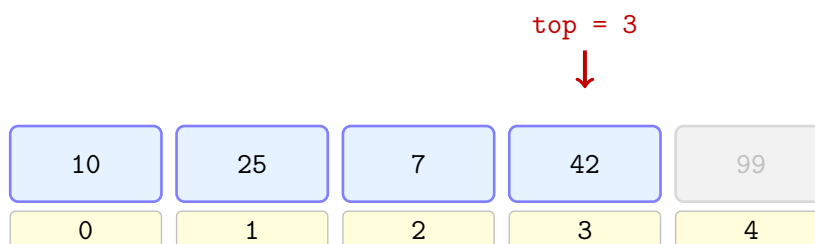
1. `isFull()`? `top(3) == capacity-1(4)?` → false, proceed
2. `top++` → `top = 4`
3. `data[top] = 99`



Stack is now FULL

pop(&outValue) — Step by Step

1. `isEmpty()`? `top == -1?` → false, proceed
2. `outValue = data[top]` → `outValue = 99`
3. `top--` → `top = 3`



99* (*stale — no longer “in” the stack*)

Pro-Tip

Notice that `pop` does **not** erase `data[4]`. The 99 is still physically sitting in memory — it’s just logically outside the stack because `top` no longer points to it. If you `push` again, that slot gets silently overwritten. This is normal and expected behavior for array-based stacks.

4 Array-Based Implementation

4.1 The Robust C Implementation

The implementation below uses a **dynamically resizable array** rather than a fixed-size one, and reports errors through return values instead of sentinel values. Every design choice here is deliberate — Section 5 explains *why*.

Listing 1: DynamicStack — Full Implementation

```

1 #include <stdio.h>
2 #include <stdlib.h>

```

```
3 #include <stdbool.h>
4
5 typedef struct {
6     int *data;
7     int top;
8     int capacity;
9 } DynamicStack;
10
11 bool initStack(DynamicStack *s, int initialCapacity) {
12     if (initialCapacity <= 0) return false;
13     s->data = (int *)malloc(initialCapacity * sizeof(int));
14     if (s->data == NULL) return false;
15     s->top = -1;
16     s->capacity = initialCapacity;
17     return true;
18 }
19
20 bool isEmpty(const DynamicStack *s) { return s->top == -1; }
21 bool isFull(const DynamicStack *s) { return s->top == s->capacity - 1; }
22
23 static bool resizeStack(DynamicStack *s) {
24     int newCapacity = s->capacity * 2;
25     int *newData = (int *)realloc(s->data, newCapacity * sizeof(int));
26     if (newData == NULL) return false;
27     s->data = newData;
28     s->capacity = newCapacity;
29     return true;
30 }
31
32 bool push(DynamicStack *s, int value) {
33     if (isFull(s)) {
34         if (!resizeStack(s)) return false;
35     }
36     s->top++;
37     s->data[s->top] = value;
38     return true;
39 }
40
41 bool pop(DynamicStack *s, int *outValue) {
42     if (isEmpty(s)) return false;
43     *outValue = s->data[s->top];
44     s->top--;
45     return true;
46 }
47
48 void freeStack(DynamicStack *s) {
49     if (s->data != NULL) { free(s->data); s->data = NULL; }
50     s->top = -1;
51     s->capacity = 0;
52 }
```

4.2 Linked-List-Based Stack Implementation

A stack can also be implemented using a singly linked list. The `top` of the stack maps directly to the head of the list — `push` = insert at head, `pop` = delete from head. This eliminates fixed-capacity and resizing concerns entirely.

Listing 2: ListStack — Linked-List Implementation

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4
5 typedef struct Node {
6     int data;
7     struct Node *next;
8 } Node;
9
10 typedef struct {
11     Node *top;
12     int size;
13 } ListStack;
14
15 bool initListStack(ListStack *s) {
16     s->top = NULL;
17     s->size = 0;
18     return true;
19 }
20
21 bool isEmptyList(const ListStack *s) {
22     return s->top == NULL;
23 }
24
25 bool pushList(ListStack *s, int value) {
26     Node *newNode = (Node *)malloc(sizeof(Node));
27     if (newNode == NULL) return false;
28     newNode->data = value;
29     newNode->next = s->top;
30     s->top = newNode;
31     s->size++;
32     return true;
33 }
34
35 bool popList(ListStack *s, int *outValue) {
36     if (isEmptyList(s)) return false;
37     *outValue = s->top->data;
38     Node *temp = s->top;
39     s->top = s->top->next;
40     free(temp);
41     s->size--;
42     return true;
43 }
44
45 void freeListStack(ListStack *s) {
46     Node *curr = s->top;
```

```
47 while (curr != NULL) {
48     Node *temp = curr;
49     curr = curr->next;
50     free(temp);
51 }
52 s->top = NULL;
53 s->size = 0;
54 }
```

Key differences from the array version:

- `pushList` is *always* $O(1)$ — no resize checks, no amortized cost.
- `popList` is *always* $O(1)$ — no shifting, no resize.
- Memory overhead per element: one `int` + one pointer (8 bytes on 64-bit) vs. just `int` for the array.
- Cache behavior: linked-list nodes are scattered across heap memory; the array version enjoys spatial locality.

5 Pitfalls While Solving

5.1 The Underflow Dilemma

Common Pitfall — Returning -1 on Underflow

A tempting but flawed design is to have `pop()` return `-1` when the stack is empty (underflow). The problem: **what if `-1` is a legitimate value the caller actually pushed?** The caller now cannot distinguish “the stack legitimately contained `-1`” from “the stack was empty and this is an error code.” The function has silently corrupted its own error-signaling channel.

The fix used above separates **status** from **data**:

- `pop()` returns a `bool` — `true` means the operation succeeded, `false` means it failed (empty stack).
- The actual popped value is written into `*outValue`, a pointer supplied by the caller.

This way, *every* `int` value — including `-1`, `0`, or `INT_MIN` — can be safely stored and retrieved without ambiguity.

5.2 The Top Pointer Index (-1 vs 0)

Common Pitfall — Where Should `top` Start?

Students often debate whether `top` should start at `-1` or `0`. This isn’t arbitrary — it changes the meaning of every operation.

With `top = -1` on an empty stack, `top` is defined as **the index of the current highest element** (not “the next free slot”). This single convention dictates the entire control flow:

- **Push** = increment first, *then* insert $\rightarrow s->top++;$ `s->data[s->top] = value;`
- **Pop** = read first, *then* decrement $\rightarrow *outValue = s->data[s->top];$ `s->top--;`
- **isEmpty** becomes a trivial check: `top == -1`.

If you instead started `top` at 0 meaning “next free slot,” you’d have to insert-then-increment for push and decrement-then-read for pop — workable, but it’s a different (and slightly more error-prone) convention, since `isEmpty` and `isFull` checks stop lining up as cleanly with the array bounds.

5.3 Pass-by-Value vs Pass-by-Reference

Common Pitfall — Passing the Struct by Value

Every function above takes `DynamicStack *s` — a **pointer** to the struct — not `DynamicStack s`. This is not a style preference.

In C, function arguments are always passed **by value**. If `push` were written as `bool push(DynamicStack s, int value)`, the function would receive a brand-new, localized **copy** of the entire struct. Any change made to `s.top` or `s.data` inside that function would modify only the copy — the original stack sitting in `main()` would be completely untouched, and the caller would see no effect whatsoever. Passing a pointer instead means the function operates directly on the caller’s actual memory, so mutations persist after the function returns.

5.4 Static vs Dynamic Scaling

Common Pitfall — Fixed Array Overflow

A naive stack backed by a fixed array, e.g. `int arr[100];`, works fine until the 101st `push()` — at which point it silently overflows the buffer, corrupting adjacent memory (or crashing) with no warning.

The implementation above avoids this entirely with `malloc` and `realloc`:

- `initStack` allocates the initial buffer on the heap, sized by the caller.
- `push` checks `isFull()` before inserting. If full, it calls `resizeStack`, which **doubles the capacity** via `realloc` before continuing.

This amortized-doubling strategy means the stack can grow to accommodate however much data is pushed, and the average cost of a `push` remains $O(1)$ amortized, even though occasional resizes are $O(n)$.

6 Debugging & Coding Tips

Avoiding Segmentation Faults

- **Always** call `isEmpty(s)` before calling `pop()` or before dereferencing `s->data[s->top]` directly. Popping from an empty stack and then reading `data[-1]` is undefined behavior — it will not politely crash every time, which makes it dangerous.
- Similarly, check `isFull(s)` (or trust `push()` to handle resizing) before writing to `data[top+1]` — never write past `capacity - 1` manually.
- After calling `initStack`, always check its `bool` return value. If `malloc` fails, `s->data` will be `NULL`, and any subsequent access will crash.

Avoiding Memory Leaks

- Every `initStack` (which calls `malloc`) must be paired with exactly one `freeStack` (which calls `free`) once the stack is no longer needed.
- If `resizeStack` is called many times, remember that `realloc` may move the block in memory — always use the pointer `realloc` returns (`newData`), never assume `s->data` is still valid until you reassign it, which the implementation above does correctly.
- Set pointers to `NULL` after freeing them (as `freeStack` does with `s->data = NULL;`). This turns any accidental use-after-free into a predictable, debuggable null-pointer crash instead of silent memory corruption.

Pro-Tip

When debugging stack logic, add a temporary `printStack()` helper that walks `data[0]` through `data[top]` and prints each value. Most “off-by-one” bugs in stack code become obvious the moment you can see `top` and the array contents side by side.

7 Practice Problems

7.1 Problem 1 — Reverse a String Using a Stack

Given a string, reverse it using stack operations only (no direct indexing tricks like `strrev`).

Approach: Push every character of the string onto a stack one at a time. Then pop characters off one at a time, writing each into a new string (or back into the original buffer) — since the stack is LIFO, the last character pushed (originally the end of the string) comes off first, naturally producing the reversal.

7.2 Problem 2 — Check for Balanced Parentheses

Given a string containing `(, ), {, }, [, ]`, determine whether every opening bracket has a matching, correctly-nested closing bracket.

High-level algorithm:

1. Initialize an empty stack.
2. Scan the string left to right, one character at a time.
3. If the character is an **opening** bracket (`(, {, [`), push it onto the stack.
4. If the character is a **closing** bracket (`), }, ]`):
 - If the stack is empty, the string is **unbalanced** — there is nothing to match against. Stop.
 - Otherwise, pop the top element and check whether it is the correct matching opening bracket for this closing bracket (e.g., `)` must match `(`). If it doesn't match, the string is **unbalanced**. Stop.
5. After processing the entire string, check the stack:
 - If it is **empty**, the string is **balanced**.
 - If any elements remain, there are unmatched opening brackets — the string is **unbalanced**.

7.3 Problem 3 — Evaluate a Postfix Expression

Given a postfix (Reverse Polish Notation) expression such as `5 1 2 + 4 * + 3 -`, compute its numeric result using a stack.

Approach (algorithm sketch):

1. Initialize an empty stack of integers.
2. Scan the expression token by token (space-separated).
3. If the token is a **number**, push it onto the stack.
4. If the token is an **operator** (`+`, `-`, `*`, `/`):
 - Pop the top two values off the stack. Call them `b` (popped first) then `a` (popped second) — note the order matters for `-` and `/`.
 - Compute `result = a <operator> b`.
 - Push `result` back onto the stack.
5. After the final token is processed, the single value remaining on the stack is the answer.

Pro-Tip

Postfix evaluation is the classic proof that stacks eliminate the need for parentheses and operator-precedence parsing entirely — the *order of the tokens themselves* already encodes the correct evaluation order.

8 Debugging Scenarios

The following five problems illustrate the most common real-world bugs students encounter when implementing stacks in C. Each shows the buggy code, the symptom, the root cause, and the fix.

8.1 Problem 1: Pass-by-Value Trap

Listing 3: Buggy — struct passed by value

```
1 void push(DynamicStack s, int value) {
2     if (s.top == s.capacity - 1) return;
3     s.top++;
4     s.data[s.top] = value;
5 }
6
7 int main() {
8     DynamicStack myStack;
9     initStack(&myStack, 10);
10    push(myStack, 42);
11    printf("Top index is: %d\n", myStack.top); // Output is still -1!
12 }
```

Symptom: The code compiles without warnings, but the stack in `main()` always appears empty.

Culprit: The `push` function accepts the stack by *value*. C makes a temporary copy of `myStack`. The copy gets updated, but as soon as `push` returns, the copy is destroyed and the original stack in `main` remains untouched.

Fix: Change the signature to `void push(DynamicStack *s, int value)` and use the arrow operator (`s->top`).

8.2 Problem 2: Wrong Top Convention

Listing 4: Buggy — top points to next free slot

```
1 // Assume top is initialized to 0 (pointing to the next available slot)
2 void push(DynamicStack *s, int value) {
3     s->data[s->top++] = value; // Inserts, then increments
4 }
5
6 int pop(DynamicStack *s) {
7     if (s->top == 0) return -1;
8     return s->data[s->top--]; // Decrements AFTER returning!
9 }
```

Symptom: If a user pushes 10 and 20, the first `pop()` returns completely random garbage memory instead of 20.

Culprit: Because `top` points to the *next available empty slot*, `s->data[s->top]` is uninitialized memory. The code returns this garbage, and *then* decrements `top`.

Fix: The `pop` function must decrement first when using this convention: `return s->data[--s->top];`

8.3 Problem 3: Realloc Overwrite

Listing 5: Buggy — realloc overwrites the only pointer

```
1 bool resizeStack(DynamicStack *s) {
2     int newCapacity = s->capacity * 2;
3     // DANGER ON THIS LINE:
4     s->data = (int *)realloc(s->data, newCapacity * sizeof(int));
5
6     if (s->data == NULL) return false;
7
8     s->capacity = newCapacity;
9     return true;
10 }
```

Symptom: The code usually works perfectly. But on low-memory systems, a crash completely destroys the program and leaks the entire existing stack memory.

Culprit: If `realloc()` fails to find new memory, it returns `NULL`. By assigning the result directly back to `s->data`, the code overwrites the only pointer it had to the existing stack data. The old data is now orphaned (a memory leak) and `s->data` is fatally broken.

Fix: Always assign `realloc` to a temporary pointer first: `int *temp = realloc(s->data, ...); if (temp != NULL) s->data = temp;`

8.4 Problem 4: Uninitialized Pointer

Listing 6: Buggy — pointer declared but not allocated

```
1 int main() {  
2     DynamicStack *myStack; // Pointer declared, but not allocated!  
3  
4     initStack(myStack, 10); // CRASH  
5     push(myStack, 50);  
6 }
```

Symptom: Immediate Segmentation Fault (Core Dumped) the moment the program runs.

Culprit: The student declared a *pointer* to a stack, but never allocated memory for the struct itself. When `initStack` tries to do `s->top = -1`;, it is writing to a random, unowned memory address.

Fix: Either declare the struct normally and pass its address (`DynamicStack myStack; initStack(&myStack, 10);`) OR allocate the struct first (`DynamicStack *myStack = malloc(sizeof(DynamicStack));`).

8.5 Problem 5: Read-Before-Check

Listing 7: Buggy — array access before bounds check

```
1 int pop(DynamicStack *s) {  
2     int value = s->data[s->top]; // Dangerous read!  
3     if (s->top == -1) {  
4         return -1;  
5     }  
6     s->top--;  
7     return value;  
8 }
```

Symptom: Random crashes or memory corruption when trying to pop an empty stack, even though there is an “underflow check” right below it.

Culprit: The code attempts to read `s->data[s->top]` *before* checking if the stack is empty. If `top` is `-1`, the program evaluates `s->data[-1]`, accessing memory outside the array bounds.

Fix: The bounds check (`if (s->top == -1)`) MUST happen before any array access.

9 Competitive Programming Tips

If you want to solve stack problems fast, here are the key insights.

9.1 Know When to Reach for a Stack

You need a stack when the problem involves **nested structure** or **reversing dependencies**. The telltale signs:

- You are scanning left to right and need to match later elements to earlier ones (parentheses, tags, intervals).
- You need the “most recent” thing that satisfies a condition (nearest smaller element, undo, backtracking).
- The order of processing should be the reverse of the order of discovery (DFS, postfix evaluation).

If you see any of these, stop thinking about arrays and start thinking about a stack.

9.2 The Nearest Smaller Element Pattern

This is the single most important stack pattern. It appears in **every** stack-heavy contest (monotonic stack problems).

Nearest Smaller to Left — $O(n)$ using a monotonic stack

```
1 int* nearestSmallerLeft(int arr[], int n) {
2     int *result = malloc(n * sizeof(int));
3     // Stack stores INDICES, not values
4     int stack[100000], top = -1;
5
6     for (int i = 0; i < n; i++) {
7         // Pop while stack top is >= current element
8         while (top >= 0 && arr[stack[top]] >= arr[i])
9             top--;
10        // If stack is empty, no smaller element on left
11        result[i] = (top >= 0) ? stack[top] : -1;
12        stack[++top] = i;    // Push current index
13    }
14    return result;
15 }
```

Why it works: The stack maintains indices of elements in increasing order. When we see `arr[i]`, any element `>= arr[i]` on the stack can never be the “nearest smaller” for any future element (because `arr[i]` is both smaller and closer). So we pop them — they are useless. This is the key insight: **the stack only keeps candidates that are still useful**.

Variants: nearest smaller to right (scan right to left), nearest greater (change `>=` to `<=`).

9.3 Stack + HashMap for Advanced Problems

Some problems (e.g., next greater element in a circular array) combine a stack with a hashmap. The pattern:

1. Traverse the array, pushing indices onto the stack.
2. When you find an element that resolves a pending stack entry, pop and record in a hashmap.
3. The hashmap stores “index $\rightarrow$ answer” for $O(1)$ lookup at the end.

9.4 Performance Rules for C

- **Use a fixed array as stack when the maximum size is known.** `int st[100000]; int top = -1;` is faster than `malloc` and avoids all resizing overhead. Most CF problems give constraint bounds — use them.
- **Store indices, not values.** The stack usually holds positions in the original array. You can always dereference `arr[st[top]]` to get the value, but you cannot recover the index from the value (duplicates).
- **Never use recursion for DFS when an explicit stack works.** Recursion will overflow the call stack on deep input (10^5). Build an explicit stack with `malloc` or a fixed array — it is not slower and it will not crash.

9.5 Classic Stack Problems You Must Know

1. **Valid Parentheses** — basic stack, $O(n)$.
2. **Nearest Smaller Element** (left & right) — monotonic stack, $O(n)$.
3. **Largest Rectangle in Histogram** — monotonic stack, $O(n)$, the classic hard problem that tests whether you really understand the pattern.
4. **Min Stack** — design a stack that supports `getMin()` in $O(1)$. Trick: store pairs or use a second stack.
5. **Stack with Increment Operation** — lazy propagation on a stack.
6. **Asteroid Collision** — simulation with a stack.
7. **Remove K Digits** — monotonic stack + greedy.

If you can solve all seven, you will never lose points on a stack problem on Codeforces.